COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Sept 8 2026	**DATE SUBMITTED:**	Sept 10 2026
NAME:	Regala, Joaquin Luiz B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

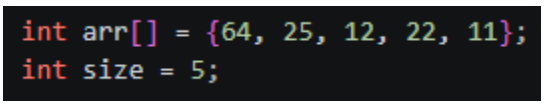
Code + Console Screenshot	
Observations	The array contains unsorted values that were used as the input for testing the sorting algorithms. The values were successfully displayed and used for sorting.

Table 7-1. Array of Values for Sort Algorithm Testing

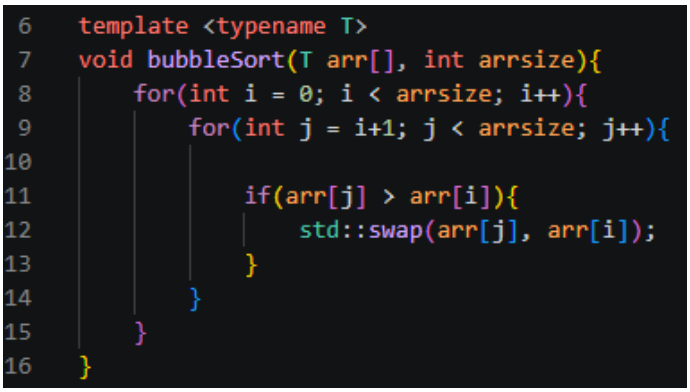
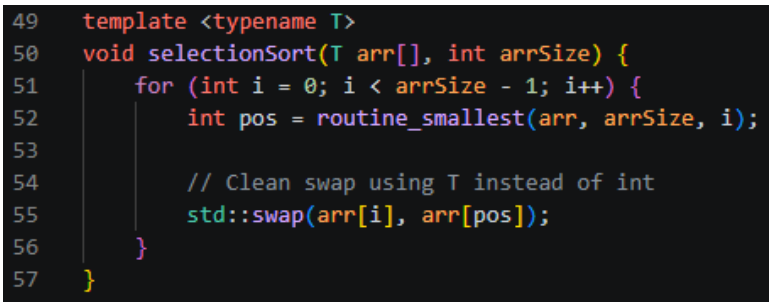
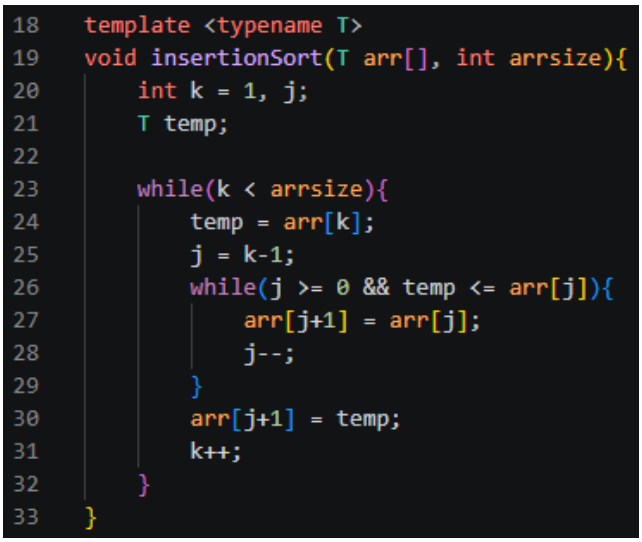
Code + Console Screenshot	
Observations	The elements are compared with adjacent elements and swapped when they are in the wrong order. The process continues until the array is sorted in ascending order.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observations	The algorithm searches for the smallest element in the unsorted portion and swaps it with the current element. The array becomes sorted after each selection.

```
49  template <typename T>
50  void selectionSort(T arr[], int arrSize) {
51      for (int i = 0; i < arrSize - 1; i++) {
52          int pos = routine_smallest(arr, arrSize, i);
53
54          // Clean swap using T instead of int
55          std::swap(arr[i], arr[pos]);
56      }
57  }
```

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	
Observations	Each element is compared with the previous elements and inserted into its correct position. The array becomes sorted gradually as each element is processed.

```
18  template <typename T>
19  void insertionSort(T arr[], int arrsize){
20      int k = 1, j;
21      T temp;
22
23      while(k < arrsize){
24          temp = arr[k];
25          j = k-1;
26          while(j >= 0 && temp <= arr[j]){
27              arr[j+1] = arr[j];
28              j--;
29          }
30          arr[j+1] = temp;
31          k++;
32      }
33  }
```

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code:

```cpp
#include <iostream>
#include "sortingAlgo.h"

int main()
{
    int A[100] = {
        1, 3, 2, 5, 1, 4, 2, 3, 1, 5,
        4, 2, 1, 3, 5, 2, 4, 1, 3, 5,
        1, 2, 3, 4, 5, 1, 2, 3, 4, 5,
        2, 1, 4, 3, 5, 1, 3, 2, 5, 4,
        3, 1, 2, 5, 4, 2, 3, 1, 5, 4,
        1, 4, 2, 3, 5, 2, 1, 4, 3, 5,
        5, 3, 1, 2, 4, 1, 5, 2, 3, 4,
        2, 4, 1, 5, 3, 3, 2, 4, 1, 5,
        4, 1, 3, 2, 5, 1, 4, 2, 3, 5,
        5, 2, 1, 4, 3, 1, 3, 5, 2, 4
    };

    int size = 100;

    // Sort the array using Bubble Sort
    bubbleSort(A, size);

    std::cout << "Sorted Array:\n";

    for (int i = 0; i < size; i++)
    {
        std::cout << A[i] << " ";

        if ((i + 1) % 20 == 0)
        {
            std::cout << "\n";
        }
    }

    // Counts the votes
    int count1 = 0;
    int count2 = 0;
    int count3 = 0;
    int count4 = 0;
    int count5 = 0;

```

```
43      for (int i = 0; i < size; i++)
44      {
45          if (A[i] == 1)
46          {
47              count1++;
48          }
49          else if (A[i] == 2)
50          {
51              count2++;
52          }
53          else if (A[i] == 3)
54          {
55              count3++;
56          }
57          else if (A[i] == 4)
58          {
59              count4++;
60          }
61          else if (A[i] == 5)
62          {
63              count5++;
64          }
65      }
66
67      std::cout << "\nVote Count:\n";
68      std::cout << "Candidate 1 - Bo Dalton Capistrano: " << count1 << " votes\n";
69      std::cout << "Candidate 2 - Cornelius Ramon Agustin: " << count2 << " votes\n";
70      std::cout << "Candidate 3 - Deja Jayla Banaga: " << count3 << " votes\n";
71      std::cout << "Candidate 4 - Lalla Brielle Yabut: " << count4 << " votes\n";
72      std::cout << "Candidate 5 - Franklin Relano Castro: " << count5 << " votes\n";
73
74      return 0;
75  }
```

Output:

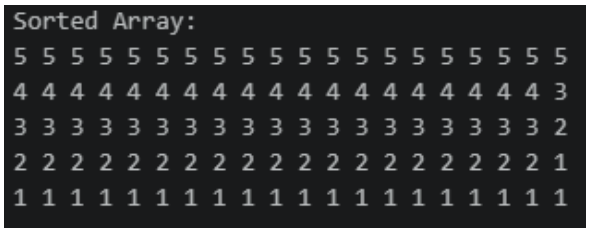
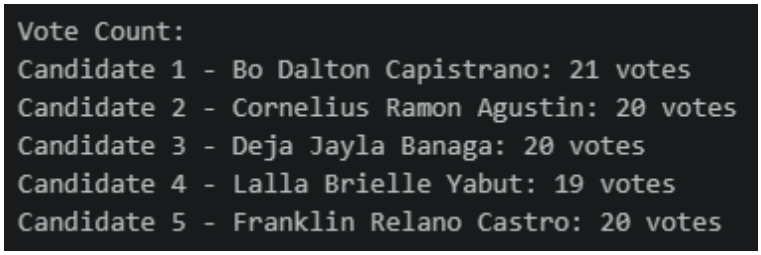
```
[Running] cd "c:\Users\regal\OneDrive\Pictures\VSCODE\CPE010\Lab7.1"
Sorted Array:
5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 2
2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

Vote Count:
Candidate 1 - Bo Dalton Capistrano: 21 votes
Candidate 2 - Cornelius Ramon Agustin: 20 votes
Candidate 3 - Deja Jayla Banaga: 20 votes
Candidate 4 - Lalla Brielle Yabut: 19 votes
Candidate 5 - Franklin Relano Castro: 20 votes

[Done] exited with code=0 in 2.112 seconds
```

Pseudocode:

```
START
        INPUT 100 VOTE VALUES FROM 1 TO 5
        CALL BUBBLE SORT(A, 100)
        DISPLAY SORTED ARRAY
        SET COUNT1, COUNT2, COUNT3, COUNT4, COUNT5 TO 0
        FOR EACH VALUE IN A DO
                IF VALUE = 1 THEN
                        COUNT1 ← COUNT1 + 1
                ELSE IF VALUE = 2 THEN
                        COUNT2 ← COUNT2 + 1
                ELSE IF VALUE = 3 THEN
                        COUNT3 ← COUNT3 + 1
                ELSE IF VALUE = 4 THEN
                        COUNT4 ← COUNT4 + 1
                ELSE IF VALUE = 5 THEN
                        COUNT5 ← COUNT5 + 1
                END IF
        END FOR
        DISPLAY VOTE COUNTS
END
```

Output Console Showing Sorted Array	
Manual Count	Candidate 1 – 21 Candidate 2 – 20 Candidate 3 – 20 Candidate 4 – 19 Candidate 5 – 20
Count Result of Algorithm	

Question: Was your developed vote counting algorithm effective? Why or why not?
Answer: Yes, the vote counting algorithm was effective because it correctly sorted the votes and counted the number of votes received by each candidate. The algorithm produced the expected results and made it easy to determine the vote count for all five candidates.

Conclusion:
In this activity, I learned how different sorting algorithms such as Bubble Sort, Selection Sort, and Insertion Sort work and how they can be used to arrange data. The procedure helped me understand how sorting can make it easier to process and count values, especially in the supplementary voting activity. I was able to successfully implement the sorting and vote-counting algorithm, although I can still improve my understanding of the algorithms and make my code more efficient. Overall, I think I performed well in this activity and gained a better understanding of basic sorting techniques and their practical applications.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.